

00:00

Alright, so today's lecture is about preparing the data. In the previous lecture we talked about cleaning data, and of course we have briefly covered these topics in the first couple of weeks of the unit. We have seen examples of cleaning and preparing data, but last week we focused more on cleaning and this week we're focusing on preparing data. By the end of this lecture we want to know how to add date, time, string, and numeric columns derived from other columns to a data frame; add summary columns; apply functions, user-defined functions, and lambda expressions to a data frame; set an index, unstack the data based on an index, and reset the index; join, merge, and concatenate data frames; and fix problems identified by the setting-with-copy warning. We're going to explore scenarios that may lead to this warning and learn how to fix them or approach them. You should also be able to describe the appropriate use of join, merge, and concatenate methods, including an inner join, a left join, and an outer join, as well as the setting-with-copy warning message and the different scenarios that may lead to it.

01:53

Before we dive into the details of data preparation, what is data preparation? The process of transforming and organizing raw data into a format suitable for analysis or machine-learning algorithms is called data preparation. It is not a new topic; we have already seen examples when we talked about pivoting and melting. Sometimes we need to restructure or reshape our data frame so that we can perform certain types of analysis or visualization. These are all examples of data preparation. Data preparation is a crucial step in the data-science workflow because the quality of the data directly affects the accuracy and reliability of any insights or models derived from it. From a broader perspective, data preparation can also include data cleaning. The purpose of cleaning is to prepare our data for analysis, improve its quality, get rid of outliers, or find out why we have outliers so we can make adjustments to our data sets. We talked about techniques for dealing with outliers in the previous lecture, so in a way we are also preparing the data. We separate cleaning and preparation into two processes because cleaning itself is a large enough topic to be discussed separately. In reality, there is overlap between preparing and cleaning, so do not be surprised if some definitions combine the two.

04:53

All right, how to add and modify columns. Let's first look at working with date-time columns. Here we have an example with a dataset called `FiresCleaned.pkl`. A PKL file is a serializable file that saves a serializable object, in this case a data frame. We can read this object back into memory, retrieve it as a data frame, and work with it. We are reading `FiresCleaned.pkl`, which is a cleaned version of the fires dataset, so it assumes that data cleaning has already been performed. We load it into memory as a data frame called `fires`, and then print the data types. The data types include fire name, which is an object or string; fire year, which is an integer; state, which is categorical; discovery date and containment date, which are date-time values; and acres burned, which is a float. We also print the first five rows using `fires.head()`, where we can see examples of values for each field. The dataset contains information about fires that have happened in the United States, including the number of acres burned.

07:37

Continuing, we extract the month of the fires using the `dt` attribute of the discovery date. Discovery date is the name of the column and one of the columns of the data frame. We extract the month and store it in a new column called `fire month`. We also add a new column containing the difference between containment date and discovery date. You can subtract the dates, so containment date minus discovery date is a column operation. Every member of the discovery-date column is subtracted from its corresponding member in containment date. Because containment date is always greater than or equal to discovery date, the result gives the number of days. We use the `dt` attribute of the resulting date difference and extract the number of days. Thus, we create two columns: `days burning`, resulting from subtracting the discovery dates from the containment dates and extracting the number of days, and `fire month`, created by accessing the month attribute of discovery date through `dt.month`.

10:50

We learned how to work with date-time columns. This is the advantage of converting date-time values. Sometimes date-time values are stored in a data frame as an object or string. If they are stored that way, you cannot use these operations. It is much easier to work with date-time objects than with strings, so it is always good practice to convert date-time strings into date-time objects. Then we can perform column opera

tions and extract months and days. This is also helpful for visualization. Sometimes we want to visualize results by week, year, or month, so converting date-time strings to date-time objects gives us more flexibility.

12:11

Working with string and numeric columns. Here is an example of applying the `title` method to capitalize the first letter of each word in the fire name column. The `fires` data frame contains information about fires, and we access the fire name column and its `str` attribute, then call the `title` method. Because fire name is a string, we can access its `str` attribute. The `title` function capitalizes the first letter of the values stored in the fire name column. We can see values such as `Power Thread` capitalized. We can also create a new string by concatenating the definite article, the word `fire`, the fire name, and the year in parentheses. The year is converted to a string using `astype(str)`, allowing it to be concatenated with the other strings. The resulting string is assigned to `full_name` for each row.

15:50

We can perform another column operation by dividing the values in `acres burned` by `days burning`. The new values are stored in a column called `acres per day`, which specifies how many acres were burned per day. We use `dropna` to remove rows with missing values so that both the numerator and denominator have values when we perform the division. We can still get a divide-by-zero situation when `days burning` is zero. In that case, we divide a number by zero and obtain infinity, which is not what we want. We should examine those situations and probably remove rows with zero days burning. We can also add summary columns. In this example, `mean days` calculates the mean of the `days-burning` column for each unique value in the `state` column. Every row with the same state receives the same mean value. For example, the rows for `CA` have the same mean days, and the rows for `NC` have another shared mean. These mean days are calculated for different states and added to the data frame as a summary column.

18:33

How to apply functions and lambda expressions. We can apply functions to columns or rows. First, consider applying functions to columns. We load a data frame from `work_data.pkl`. PKL files store serializable objects, in this case a data frame, so we deseria

lize the data frame into memory and display its first three rows. We apply the mean function to all numerical columns of the data frame, calculating means for columns such as sex, region, and work status. We can see the mean value for each column. In another example, we use the NumPy library, imported as np, and apply np.mean to a subset of the data frame rather than to every column. We select two columns and apply the mean function to the new data frame containing only those columns, obtaining the mean value for each selected column.

20:50

We can also apply the mean function to rows by specifying axis equal to one. This means the function is applied across the rows. For each row, the function takes the values from the specified columns as input and calculates their mean. The new column is called avg_rating and contains the mean values. Instead of using functions from specific modules, we can create our own functions. We define a function called convert Sex that takes a row of the data frame as input and compares the value of sex with 1 and 2. If the value is 1, the function returns male. If it is 2, it returns female. If it is neither of these values, it returns non-binary. We apply this user-defined function to the data frame with axis equal to one, so it is applied across the rows. For a row whose sex value is 2, the function returns female, and the original numeric value is replaced with the more readable value. This is useful for visualization or communication because someone may not know what the values 1 and 2 mean.

24:49

Another example uses a custom function to determine a season from the month of a game date. The function takes a row as input and checks the month of the game date. If the month is greater than 6, it assigns a season based on the current year and the next year. For example, if the date is in 2009, the result is 2009 to 2010. If the month is less than 6, it assigns the season from the previous year to the current year. The example is corrected so that when the current year is 2019, the result is 2018 to 2019. We apply the get_season function to every row of the game data frame using apply with axis equal to one, generating a new column with the corresponding season value.

02:47

Lambda expressions allow us to define and use a function at the same time. They can

n support more sophisticated operations, although that is beyond the scope of this unit. Here we focus on the basic form. A lambda expression takes x as input, calculates the sum of the values in x , and multiplies the result by 2. Lambda is a keyword, and x is the input. With axis equal to zero, the lambda expression is applied to the columns of the data frame. For the first column, 3 plus 0 equals 3, multiplied by 2 gives 6. For the second column, 4 plus 1 equals 5, multiplied by 2 gives 10. For the third column, 2 plus 5 equals 7, multiplied by 2 gives 14. This is a column-wise operation. Using axis equal to one applies the same expression across the rows. It takes a row as input, sums the row values, and multiplies the result by 2. For example, 0 plus 1 plus 2 equals 3, giving 6, and 3 plus 4 plus 5 equals 12, giving 24. The 0 and 1 specify indices, not values.

03:07

Here is another lambda example. The expression takes a row as input and checks whether work status is 1. If it is, work status is changed to full time; otherwise it is changed to part time. The expression is applied across the rows with axis equal to one. For the first row, the original work-status value is 1, so the new value is full time. A row with value 2 is changed to part time. The same process is applied to the other rows: values of 1 become full time, and values that are not 1 become part time. A separate function could have been defined instead, but when the function is short, using a lambda expression directly is more readable and easier.

34:25

Another lambda expression takes a row as input and splits the value of car name. The string is split using a space as the delimiter. If there were a comma, we could pass a comma to the split function, but here the empty space separates the string into two substrings. We obtain a list of strings and select the first one, the left-hand part. We use that value to create a brand column. Thus, the value of car name in each row is split, and the first part is assigned to brand. In other words, we extract the brands and place them in the new column, producing values such as Audi and Alfa Romeo. There is another video showing use cases of lambda expressions in more detail, and it is recommended for anyone interested.

36:59

How to work with indices or indexes. We can set a column as an index, as we have se

en before. We read a dataset into a data frame called `fires` by month and use `set_index` to set state as the index. Instead of sequential numbers such as 0, 1, and 2, the state becomes the new index. We can also specify multiple columns to form the index, such as state, fire year, and fire month. The data is then indexed by all three columns together. We can reset the index to flatten the data frame, treating the index columns as normal columns again. Resetting the index changes the data frame from the indexed shape to a regular shape. The states and years are repeated for the relevant data points, and new sequential indices are automatically generated.

39:36

We can unstack data based on different columns specified by level. Unstacking helps restructure or reshape a data frame so that the unique values of the selected level form new column headers. In the example, we select two columns from the top-five-states dataset and unstack based on state. The result contains days burning and fire count, with values corresponding separately to each state. Under each category are subcolumns for the states. The data has been unstacked based on the state level. A further example unstacks all columns based on state, so for each column—acres burned, days burning, and fire count—the different states are listed. The fire year is the index. When no particular column is specified, unstacking is performed for all columns.

04:20

In another example, only fire count is selected and unstacked based on state. A new state level is added, and the resulting values are fire counts across different years for different states. This is a subset of the previous result. Another video about unstacking is available and is strongly recommended because it provides more examples and summarizes the discussion.

44:36

How to combine data frames. We can concatenate data frames. Here we have three data frames: `shots1` contains the first two rows of the `shots` data frame; `shots2` contains data points with a particular game ID; and the third contains rows corresponding to other game IDs. We combine them with `pd.concat`, providing `shots1`, `shots2`, and `shots3`. The rows are vertically attached or concatenated. A more sophisticated example concatenates two data frames after resetting the index of the second one, dropping the old indices, and dropping the fire month and days burning columns. The two d

ata frames do not have exactly the same structure. The first data frame has all columns, while the second lacks fire month and days burning. When they are concatenated, the missing values in the second data frame's corresponding columns are represented by NaN. The two data frames can still be combined even though they do not have the same set of columns.

47:52

Here is an example of joining two data frames using an inner join. We join shots and points by game using game ID as the key. The how parameter is set to inner. Shots is the left data frame and points by game is the right data frame. We join them based on the common game ID column. To understand the operation, take each row in the left data frame and check whether its game ID appears in the right data frame. When there is a match, keep the row and add information from the right data frame. In other words, we find the intersection of the game IDs and combine the columns. Rows without matching game IDs are not included. Therefore, rows with game IDs such as 69 or 82 are excluded when they do not appear in the other data frame. Matching rows remain, and values such as total score are appended. The result is the first data frame filtered to matching rows, with total scores such as 14 for game ID 15 and 12 for game ID 30.

51:29

A left join works similarly, but every row from the left data frame is included even when there is no match in the right data frame. The result contains all rows from shots. Additional information, such as total score and player name, is added where a match exists. If a left-side game ID has no match in the right data frame, its additional values are NaN. Information from a row that exists only in the right data frame is not included. A left join can also be achieved with a left merge by specifying game ID as the column on which to merge. The first data frame is the left one and the second is the right one, and the result is the same. Another operation is an outer join. An outer join includes all rows from both data frames. Matching values are combined, while missing values are represented by NaN. Thus, rows that occur only in the left data frame and rows that occur only in the right data frame are all included. A video with examples of join, merge, and concatenation operations is also available.

58:15

The last part of the lecture deals with a specific warning called the setting-with-copy warning. In computer programming, a warning is not an error; it does not stop code execution. It may not cause a problem, but it might lead to an error in the future, so it is worth investigating how it affects a program or script. Setting-with-copy warning is an important warning when working with data. Documentation links can sometimes be confusing because they are technical, so we will keep the explanation simple. Suppose `df` equals `shots.copy()`, creating a copy of the data set. Then `df_slice` equals `df.query(...)`, extracting rows with a particular game ID. The query operation may create and return a new data frame rather than a view or reference to the original. If so, changes to `df_slice` do not affect `df`. Updating the player-name column in `df_slice` to Curry changes the slice, while the original `df` still contains Stephen Curry. This does not corrupt the original data, but we may still receive a warning because we do not know whether query returns a copy or a view internally.

01:02:56

The warning tells us to be careful because the behavior depends on how the query function is implemented in pandas, and that implementation could change. We can explicitly call `copy` after the query. This creates a copy rather than relying on the query function to return one, ensuring that changes to the resulting data frame do not affect the original. If query already returned a copy, the explicit copy only prevents the warning from being displayed. It does not solve a more sophisticated problem by itself, but it protects against a future implementation that might return a view. Without a copy, a slice may be only a pointer or reference to part of the original data frame. Changes made through the slice can then affect the original data. In that case, the data may be corrupted. Calling `copy` ensures that modifications are applied to an independent data frame.

01:06:15

Here is another example in which the data is corrupted. We create a data frame and then create a slice based on an index, extracting two rows corresponding to particular game IDs. We change the player-name value in the slice to Curry. At first this appears harmless, but the change is reflected in the original data frame as well. This happens because the slice is a view or reference rather than an independent copy. The warning tells us to be careful, since we may not know whether the operation returns a copy or a view until we test it. To prevent the problem, call the `copy` function when creati

ng the slice. Then changes to the copied slice affect only that copy and not the original data frame. The original data remains preserved and there is no data corruption.

01:10:50

Data corruption can also happen even when we do not receive a setting-with-copy warning, so we cannot rely on the warning to prevent corruption. For example, assigning `df_slice` equal to `df` makes `df_slice` a pointer or reference to `df`. Any operation carried out on `df_slice` is reflected in `df`, even though no warning is displayed. Be careful with this kind of assignment. If you want to prevent the problem, call `copy` and create an independent copy of the dataset. Otherwise, changing player names in the slice may unintentionally change them in the original data frame. The `copy` function preserves the original data. Thanks for your attention. This wraps up our week-six lecture, and see you next time.